

Serving Turns a Fitted Model into a Service

Model serving makes a fitted model available to another program.



CORE CONCEPT

HTTP Defines Requests and Responses

HTTP is a protocol for exchanging requests and responses. A request contains: A response contains a status code, headers, and optional body.

method such as GET or POST;

path such as /predict;

headers such as content type or authorization;

optional body containing input data.

200: success

400 or 422: invalid client input

RUN IT AND INSPECT THE RESULT

JSON and Validation Define the API Contract

```
from pydantic import BaseModel, Field

class PredictionRequest(BaseModel):
    age: int = Field(ge=18, le=100)
    city: str = Field(min_length=1)
```

OUTPUT

Valid JSON becomes a typed request object; invalid JSON returns field-level errors.

`{"age": "thirty-five"}` fails type and missing-city checks before model inference.

REASONING SEQUENCE

FastAPI Connects a Typed Contract to Inference

Frame

accept a typed
request;

1

Transform

convert it into the
model's row format;

2

Validate

call the loaded
pipeline;

3

Explain

return JSON-safe
values;

4

For probability 0.73 and threshold 0.50, return: The version lets logs and clients identify the producing artifact.

CORE CONCEPT

Health Checks Separate Availability from Prediction Quality

A health check is a lightweight endpoint used by operators or infrastructure. A readiness check may confirm the model loaded and required dependencies are available. It should not perform an expensive full prediction on every probe. Health checks do not measure model accuracy or data drift.

Liveness: is the process running?

Readiness: can it currently serve valid requests?

Startup: has initialization completed?

RUN IT AND INSPECT THE RESULT

Docker Packages the Application Environment

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It contains the following Dockerfile content:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["fastapi", "run", "app.py", "--port", "8000"]
```

OUTPUT

One reproducible image that starts the API on port 8000.

The host port 8000 forwards to the container service port.

CORE CONCEPT

Configuration and Secrets Belong Outside Source Code

Configuration changes application behaviour between environments. A secret grants access, such as an API key or password. Environment variables can supply configuration at runtime: MODELPATH, LOGLEVEL, PREDICTIONTHRESHOLD

Configuration changes application behaviour between environments.

A secret grants access, such as an API key or password.

Environment variables can supply configuration at runtime:

MODELPATH, LOGLEVEL,
PREDICTIONTHRESHOLD

Do not commit .env files containing real credentials.

In production, use a managed secret store or scoped instance role.

REASONING SEQUENCE

EC2 Is a Virtual Server You Must Operate

Frame

AMI:
operating-system
image;

1

Transform

instance type: CPU
and memory
capacity;

2

Validate

EBS volume:
persistent block
storage;

3

Explain

VPC and subnet:
network placement;

4

For a small CPU-only API, choose the smallest architecture-compatible instance that meets measured memory needs. Start with one instance, no load balancer, and a documented restart procedure.

RUN IT AND INSPECT THE RESULT

Security Groups Expose Only Required Traffic



```
Internet :443 -> reverse proxy -> localhost:8000 -> FastAPI  
Admin IP :22 -> EC2
```

OUTPUT

Only encrypted client traffic and restricted administration reach the instance.

The reverse proxy terminates TLS and forwards internally to FastAPI.

QUALITY GATE

Logs and Metrics Make Failures Observable



timestamp and request ID;



model version;



status code and error type;



latency;

QUALITY GATE

Cost Controls Start Before the Instance



confirm current Free Tier or credit eligibility;



estimate instance and storage cost;



choose the smallest measured capacity;



set AWS Budgets alerts;

GUIDED LAB

Guided Lab: Deploy, Verify, and Clean Up

01

**local API
contract tests**

02

**model loaded
once at startup**

03

**health and
prediction
endpoints**

04

**Docker image
build and local run**